



Análisis de Sentimiento

Módulo 4 · Código Heredado

F

Refactorización de Código Heredado:

Análisis de Sentimiento con IA

Ejercicio de Clase · Ejercicio 3 de 3 · Proyecto profesional completo con IA como copiloto

Nombre del alumno/a	
Fecha	
 Enlace GitHub	

 **Instrucciones:** En este ejercicio partirás del código heredado `InicialSentimiento.py`, un analizador de sentimientos con IA. Tu misión es refactorizarlo en módulos, escribir tests, configurar CI/CD, implementar almacenamiento de resultados en TXT y JSON, y documentar cada paso como haría un profesional.

BLOQUE 1 — El Código Heredado

A continuación tienes el código `InicialSentimiento.py`. Es funcional pero mezcla responsabilidades, carece de tests, no persiste resultados y no sigue buenas prácticas. Tu trabajo es transformarlo en un proyecto profesional.

```
# InicialSentimiento.py - CÓDIGO HEREDADO
import os, json
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI(api_key=os.getenv('OPENAI_API_KEY'))

def analizar_sentimiento_basico(texto: str) -> dict:
    # Nivel básico: solo categoría
    response = client.chat.completions.create(...)
    return { 'nivel': 'básico', 'sentimiento': response.choices[0].message.content
}

def analizar_sentimiento_intermedio(texto: str) -> dict: ...
def analizar_sentimiento_avanzado(texto: str) -> dict: ...
```



Análisis de Sentimiento



```
def analizar_sentimiento_multitexto(textos: list) -> list: ...

# --- Ejecución directa (mezclada con la lógica) ---
texto_prueba = 'El producto llegó rápido, pero la calidad no es lo que esperaba.'
print(analizar_sentimiento_basico(texto_prueba))
print(analizar_sentimiento_intermedio(texto_prueba))
```

1.1 — Identifica los problemas de diseño

Sin refactorizar todavía, lista al menos 5 problemas de diseño del código (no errores de ejecución, sino malas prácticas):

#	Problema detectado	Impacto en producción
1		
2		
3		
4		
5		

BLOQUE 2 — Modularización

🔗 *Conexión con el módulo anterior: tras analizar el código heredado, el primer paso profesional es separarlo en módulos con responsabilidad única (Principio SRP). Cada módulo debe hacer UNA sola cosa.*

2.1 — Diseña la estructura de carpetas

Define la estructura de directorios del proyecto. Indica qué archivo va en cada carpeta y por qué:

Archivo / Carpeta	Responsabilidad única	Funciones que contiene
sentimiento/cliente.py	Inicializa el cliente OpenAI	get_client()
sentimiento/analizador.py		
sentimiento/niveles.py		
sentimiento/multitexto.py		



Análisis de Sentimiento



almacenamiento/guardar.py	Persistencia de resultados	guardar_txt(), guardar_json()
almacenamiento/leer.py		
main.py	Punto de entrada	Orquesta y ejecuta

2.2 — Estructura de carpetas de almacenamiento

El proyecto debe almacenar automáticamente los resultados en carpetas separadas según el formato:

```
proyecto/
├── sentimiento/
│   ├── __init__.py
│   ├── cliente.py          # Inicialización OpenAI
│   ├── analizador.py      # Lógica de análisis
│   ├── niveles.py         # Básico / Intermedio / Avanzado
│   └── multitexto.py      # Análisis por lotes
├── almacenamiento/
│   ├── __init__.py
│   ├── guardar.py         # Guardar TXT y JSON
│   └── leer.py            # Leer histórico
├── resultados/
│   ├── txt/               # Un .txt por análisis
│   └── json/              # Un .json por análisis
├── tests/
│   ├── test_analizador.py
│   └── test_almacenamiento.py
├── docs/
├── main.py
├── requirements.txt
└── README.md
```

2.3 — Prompt para que la IA refactorice

Redacta el prompt completo que le darías a la IA para realizar la refactorización. Incluye los 5 elementos del prompt profesional:

Escribe tu prompt aquí:

2.4 — Escribe el código refactorizado



Análisis de Sentimiento



Pega aquí el código de cada módulo tras la refactorización:

```
# sentimiento/analizador.py
# (escribe aquí tu código)
```

```
# sentimiento/niveles.py
# (escribe aquí tu código)
```

```
# almacenamiento/guardar.py
# (escribe aquí tu código)
```

2.5 — Formato de los archivos de salida

El módulo guardar.py debe producir dos archivos por cada análisis: uno en TXT (legible) y otro en JSON (estructurado para sistemas). Ejemplo del nombre de archivo esperado:

```
# Formato de nombre de archivo:
# resultados/txt/ analisis_2025-04-08_143022.txt
# resultados/json/ analisis_2025-04-08_143022.json

# Contenido TXT (entrada + salida legible):
# =====
# ANÁLISIS DE SENTIMIENTO — 2025-04-08 14:30:22
# =====
# TEXTO ANALIZADO:
# El producto llegó rápido, pero la calidad no es lo que esperaba.
#
# RESULTADO BÁSICO: negativo
# RESULTADO INTERMEDIO: negativo | polaridad: -0.4 | intensidad: media
# JUSTIFICACIÓN: Mezcla de aspectos positivos (rapidez) y negativos (calidad)

# Contenido JSON (estructura completa para APIs):
# { 'timestamp': '...', 'texto': '...', 'basico': {...},
#   'intermedio': {...}, 'avanzado': {...} }
```

BLOQUE 3 — Tests Unitarios

 **Regla profesional:** nunca se sube código a producción sin tests. En este bloque escribirás tests para cada módulo usando pytest, incluyendo tests para el módulo de almacenamiento (TXT y JSON).



3.1 — Prompt para que la IA genere los tests

Escribe el prompt que usarías para que la IA genere los tests unitarios. Recuerda pedir: casos felices, casos borde y casos de error:

Escribe tu prompt aquí:

3.2 — Escribe al menos 6 casos de test

Crea el archivo tests/test_analizador.py con mínimo 6 casos de prueba. Planifícalos antes de escribir el código:

Nombre del test	Tipo (feliz/borde/error)	Input / condición	Resultado esperado
test_1_			
test_2_			
test_3_			
test_4_			
test_5_			
test_6_almacenamiento_txt	feliz	Resultado válido	Archivo .txt creado en resultados/txt/
test_7_almacenamiento_json	feliz	Resultado válido	Archivo .json creado en resultados/json/

3.3 — Código de los tests

```
# tests/test_analizador.py
import pytest
from sentimiento.analizador import analizar_sentimiento_basico
from almacenamiento.guardar import guardar_resultado
import os, json

# Escribe aquí tus tests
```

```
def test_sentimiento_positivo():
    # ...
    pass

def test_guardar_txt_crea_archivo():
    # Verifica que el archivo TXT se crea en resultados/txt/
    pass

def test_guardar_json_crea_archivo():
    # Verifica que el archivo JSON se crea en resultados/json/
    pass
```

BLOQUE 4 — Pipeline CI/CD con GitHub Actions

🔗 *Conexión con el módulo de Pipeline: con el código modularizado y los tests escritos, el último paso es automatizar la validación. Cada vez que hagas git push, GitHub Actions ejecutará linter, tests y análisis de seguridad automáticamente.*

4.1 — Diseña el fichero .github/workflows/ci.yml

Indica qué debe hacer cada fase del pipeline y qué herramienta usarías:

Fase del pipeline	¿Qué debe hacer?	Comando / herramienta
1. Checkout del código		actions/checkout@v4
2. Configurar Python		actions/setup-python@v5
3. Instalar dependencias		pip install -r requirements.txt
4. Ejecutar linter		flake8 / ruff
5. Tests unitarios		pytest
6. Cobertura de tests		pytest --cov
7. Verificar carpetas resultados		python scripts/check_folders.py
8. Análisis de seguridad		bandit

4.2 — Escribe el fichero ci.yml completo



Análisis de Sentimiento



```
# .github/workflows/ci.yml
name: CI Pipeline — Análisis de Sentimiento

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      # Completa aquí cada step del pipeline
```

4.3 — ¿Qué ocurre si un test falla en el pipeline?

Explica el flujo: ¿se bloquea el merge? ¿Quién recibe la notificación? ¿Cómo se depura el error?

Escribe tu respuesta aquí:

BLOQUE 5 — Almacenamiento de Resultados (TXT y JSON)

 *Requisito profesional: los resultados de entrada y salida deben persistir automáticamente en carpetas separadas. Carpeta resultados/txt/ para lectura humana y resultados/json/ para integración con sistemas externos.*

5.1 — Diseña el módulo almacenamiento/guardar.py

Completa la lógica de persistencia. El módulo debe crear las carpetas si no existen, guardar con timestamp y no sobrescribir archivos anteriores:

```
# almacenamiento/guardar.py
import os, json
from datetime import datetime
from pathlib import Path
```

```

CARPETA_TXT = Path('resultados/txt')
CARPETA_JSON = Path('resultados/json')

def _crear_carpetas():
    CARPETA_TXT.mkdir(parents=True, exist_ok=True)
    CARPETA_JSON.mkdir(parents=True, exist_ok=True)

def guardar_resultado(texto_entrada: str, resultados: dict) -> dict:
    """Guarda en TXT y JSON. Devuelve las rutas de los archivos creados."""
    _crear_carpetas()
    timestamp = datetime.now().strftime('%Y-%m-%d_%H%M%S')
    nombre_base = f'análisis_{timestamp}'
    # (escribe aquí tu código para TXT y JSON)
    return { 'txt': str(ruta_txt), 'json': str(ruta_json) }

```

5.2 — Diseña el módulo almacenamiento/leer.py

El módulo leer.py debe permitir recuperar el historial de análisis. Define sus funciones principales:

Función	Descripción	Retorna
listar_analisis()	Lista todos los archivos guardados	list[str]
leer_json(nombre)	Lee un análisis en formato JSON	dict
leer_txt(nombre)	Lee un análisis en formato TXT	str
buscar_por_fecha(fecha)	Filtra por fecha	list[str]

5.3 — Escribe el código de leer.py

```

# almacenamiento/leer.py
from pathlib import Path
import json

CARPETA_TXT = Path('resultados/txt')
CARPETA_JSON = Path('resultados/json')

def listar_analisis() -> list:
    # (escribe aquí tu código)
    pass

def leer_json(nombre: str) -> dict:
    # (escribe aquí tu código)
    pass

```




BLOQUE 6 — Documentación Profesional

📌 *Un proyecto sin documentación es un proyecto muerto. En este bloque crearás el README y la carpeta docs/ siguiendo estándares profesionales. Incluye la documentación del sistema de almacenamiento.*

6.1 — Estructura de la carpeta docs/

Define qué archivos crearás dentro de docs/ y qué contendrá cada uno:

Archivo en docs/	Contenido / propósito
arquitectura.md	Diagrama de módulos y flujo de datos
almacenamiento.md	Documentación del sistema TXT y JSON
api_referencia.md	Referencia de funciones públicas

6.2 — Escribe el README.md del proyecto

El README debe incluir: descripción, instalación, uso, estructura de carpetas (incluyendo resultados/), cómo ejecutar los tests y badge del pipeline CI:

```
# README.md
# Análisis de Sentimiento con IA

## Descripción

## Instalación
```bash
```

## Uso

## Estructura del proyecto
```
```

## Almacenamiento de resultados
# Explica aquí cómo se guardan los TXT y JSON
```



```
## Tests
```bash
```

## Pipeline CI

## Licencia
```

6.3 — Commits semánticos

Indica qué mensaje de commit escribirías para cada cambio. Usa el formato: tipo(scope): descripción corta

| Tipo de cambio | Tu mensaje de commit | ¿Por qué este mensaje? |
|----------------------|----------------------|------------------------|
| Modularización | | |
| Añadir tests | | |
| CI/CD | | |
| Almacenamiento TXT | | |
| Almacenamiento JSON | | |
| Documentación README | | |
| Corrección de bug | | |

BLOQUE 7 — Resultado Final de la Empresa

El resultado final del proyecto debe incluir la interfaz gráfica (GUI) desarrollada con Tkinter, que muestra los análisis en tiempo real y confirma el guardado automático de resultados.

Análisis de Sentimiento - Local

ANÁLISIS DE SENTIMIENTO - LOCAL

Texto a analizar

Me encanta este producto. La calidad es excelente y el servicio al cliente es maravilloso. Definitivamente lo recomendaría a mis amigos.

ANALIZAR SENTIMIENTO

LIMPIAR

GUARDAR

Resultados por Nivel

Análisis Detallado

Justificación & Recomendación

Historial

| Nivel | Sentimiento | Polaridad | Intensidad |
|---|-------------|-----------|------------|
| ☒ Básico | POSITIVO | 90.99% | - |
| ☐ Intermedio | POSITIVO | 0.9 | ALTA |
| ☐ Avanzado | POSITIVO | 0.9 | - |

¿Qué significa la polaridad?

☒ POSITIVA (+0.00 a +1.00): El texto expresa emociones positivas

☐ NEGATIVA (-1.00 a -0.00): El texto expresa emociones negativas

☐ NEUTRAL (0.00): El texto no muestra emociones fuertes

☒ Análisis completado y guardado automáticamente

La interfaz debe mostrar:

| Componente de la interfaz | Descripción | Obligatorio |
|------------------------------|---|-------------|
| Área de texto de entrada | Campo para introducir el texto a analizar | Sí |
| Botón Analizar Sentimiento | Lanza el análisis en los 3 niveles | Sí |
| Botón Limpiar | Limpia todos los campos | Sí |
| Botón Guardar | Guarda manualmente en TXT y JSON | Sí |
| Pestaña Resultados por Nivel | Tabla con Básico / Intermedio / Avanzado | Sí |



Análisis de Sentimiento



| | | |
|------------------------------------|--|----|
| Pestaña Análisis Detallado | Emociones y polaridad extendida | Sí |
| Pestaña Justificación | Explicación del análisis + recomendación | Sí |
| Pestaña Historial | Lista de análisis guardados | Sí |
| Panel ¿Qué significa la polaridad? | Leyenda explicativa | Sí |
| Barra de estado | Confirma guardado automático | Sí |

📌 El guardado automático debe activarse tras cada análisis exitoso. La barra de estado debe mostrar: '✓
Análisis completado y guardado automáticamente' con la ruta del archivo generado.

BLOQUE 8 — Reflexión Integrada

📌 Reflexión final: Describe el flujo profesional completo que acabas de implementar. ¿Qué aporta cada paso (análisis → modularización → tests → CI/CD → almacenamiento → docs)? ¿Qué hubiera pasado si te saltas alguno?

8.1 — Flujo profesional completo

Describe con tus palabras qué aprendiste en cada fase y por qué el orden importa:

Escribe tu reflexión aquí:

8.2 — ¿Cómo usaste la IA como copiloto?

Para cada bloque, indica en qué momento pediste ayuda a la IA y cómo validaste su respuesta antes de aplicarla:

| Bloque | ¿Para qué usé la IA? | ¿Cómo verifiqué su respuesta? |
|---------------------------|----------------------|-------------------------------|
| Bloque 2 — Modularización | | |
| Bloque 3 — Tests | | |
| Bloque 4 — CI/CD | | |



Análisis de Sentimiento



| | | |
|---------------------------|--|--|
| Bloque 5 — Almacenamiento | | |
| Bloque 6 — Documentación | | |

8.3 — Autoevaluación

Marca con una X tu nivel de confianza al finalizar el ejercicio:

| Nivel | Descripción | ¿Qué necesitarías repasar? |
|----------------------------|-------------------------|----------------------------|
| ☐ 1 | Necesito más práctica | |
| ☐ 2 | Lo entiendo con ayuda | |
| ☐ 3 | Lo aplico con seguridad | |
| ☐ 4 | Podría enseñarlo | |

BLOQUE 9 — Reflexión MemoriaFinal.md

En una carpeta de MemoriaFinal, señalar en un archivo llamado [memoriaFinal.md](#) donde explicareis el porqué el código original está mal configurado y que cambios realizasteis